Calyx: A Compiler That Demands the Effect Contract in Agent Workflows

Dalton Fontes

https://github.com/daltonfontes/calyx

AI disclosure. Calyx was conceived and directed by the author, who read the related work and reviewed every change; the code, the experiments and this text were written with Claude (Anthropic) as a coding assistant.

Abstract

LLM agent workflows call models and tools whose effects leave the run: they pay, send e-mail, edit repositories and wait for people. Agent frameworks such as LangGraph and durable execution engines such as Temporal resume an interrupted run, but leave the contract of each effect to the programmer: whether it may be repeated, under which key, and what to do when nobody knows whether it happened. When that code is missing, the result is a second payment, a repeated e-mail, a lost update.

Calyx is a workflow language in which every tool declares its effect (`read`, `write` with an idempotency key, or `write once` with a policy for uncertain outcomes), and the compiler derives parallelism and checks from the dependency graph. A program that can duplicate an effect, lose an update or leave two writes unordered is rejected or warned about before it runs; the runtime records every call in a journal and resumes without redoing work. We state the guarantees as three theorems under five explicit hypotheses, prove the per-call protocols in Lean, and check whole programs exhaustively on a bounded model, which also produces a counterexample for each hypothesis dropped.

Against LangGraph and Temporal on the same workflows, Calyx gets every crash, wait and concurrency scenario right by default; the baselines get them right only with hand-written care. On LIMBO, an external benchmark of duplicated side effects, Calyx programs match the best frontier models under the tools' native contracts (76% exactly-once success versus 74–79%) and reach 100% when every write accepts a key. The guarantee lives in the tool contract; Calyx makes the contract mandatory.

1 Introduction

A support agent receives a refund request, proposes an amount, a person approves it, the agent pays and e-mails the customer. If the process dies after the payment is made but before the result is recorded, does the resumed run pay again? If it dies with the e-mail in flight, does it send it again? If the approval arrives after the deadline but before anyone resumes the run, does it count?

In current frameworks these questions have answers, but the answer is code the programmer writes, or forgets to write, with nothing to remind them. We measured it (§6): with LangGraph's defaults, the crash that Calyx survives produces two payments; with Temporal, which records every step in a durable history, an effect in flight at the time of the crash is repeated.

The thesis of this work is that **the contract of an effect belongs in the tool's declaration, and the compiler should demand it**. The runtime mechanisms already exist: per-call journals (Temporal [21], Restate [17]), idempotency keys [9, 18] and verify-before-retry [13]. There is also independent evidence that exactly-once behaviour depends on the tool contract rather than the model: LIMBO [12] finds that offering an idempotency key on every write cuts duplicates sharply, and that for failures a read-back cannot resolve (late commits, redelivery) only the contract helps. What is missing is something that **requires** the contract.

Our contributions:

1. A language for agent workflows (§3) whose execution graph is implicit in data dependencies, and in which every tool declares its effect and what to do with an uncertain outcome.
2. Compiler checks (§4) that use those declarations to reject programs that can duplicate an effect, lose an update, leave writes unordered, wait forever, or repeat a payment on every turn of a loop; and a formal core with three theorems, proved in Lean for one call and checked exhaustively on a bounded model for whole programs.
3. A runtime (§5) with a per-call journal that resumes without redoing work and carries the idempotency key to the tool's server. The mechanisms are not new; what is new is a compiler that demands the declarations they rely on.
4. An evaluation (§6) against LangGraph and Temporal, with the full crash matrix, deadlines, shared memory, runtime cost, a real-bug study with an unfavourable result, and an external benchmark (LIMBO).

2 The Problem

The bugs we target are not business-logic bugs but **state** bugs: what happens to effects when a run is parallel, interrupted, retried or concurrent. Table 1 lists seven categories, taken from the issue study of §6.6.

Category	Example
Repeated effect	A retried task pays again
Concurrent write	Two parallel branches write the same field
Lost update	Two runs read a balance, add, write back
Wait	An approval with no deadline; an answer applied twice
Agent loop	The agent repeats the same call until the limit
Effect order	The confirmation e-mail goes out before the payment
Recovery	Resuming redoes work or restores inconsistent state

Table 1: State bugs in agent workflows.

What these bugs share is that the program does not say what needs saying: that the payment cannot be repeated without a key, that the e-mail needs a rule for when nobody knows whether it went out, that the e-mail comes after the payment. In Python this information has nowhere to live where a tool can check it.

3 The Language

A Calyx program declares models, tools, typed prompts and graphs. The syntax resembles Python; the semantics is dataflow.

```
tool refund(request: Text, order: Text,
            amount: Float) -> Unit:
    effect write
    idempotency_key request
    checks OrderState

tool email(to: Text, subject: Text,
           body: Text) -> Unit:
    effect write once
    timeout 10 s
    on_uncertain verify(email_sent(to, subject))

graph handle_refund(request: Text, order: Text,
                    message: Text) -> Text:
    found = get_order(order)
    proposal = gemini(decide(found, message))

    paid = refund(request, order, proposal.amount):
        requires state.status == Delivered
        requires state.refunded + proposal.amount
                 <= state.total

    body = gemini(reply(found, proposal.amount))
    notice = email(found.email, "Refund {order}", body)
    notice after paid

    return "refunded {proposal.amount}"
```

Listing 1: The refund in Calyx. No line says what runs in parallel or what to do after a crash.

Implicit graph. Each `name = value` is a step, and a step depends on the steps it uses. The compiler orders steps by dependency and runs independent ones in parallel (in Listing 1, `reply` may run alongside `refund`). `after` adds an order without data.

Declared effects. A tool is `read`, `write` or `write once`. A `write` may be repeated if it has an idempotency key. A `write once` is never repeated by itself and must say what to do when its outcome is uncertain: `pause` (a person decides when resuming), `accept_loss`, or `verify(f(...))`, where `f` is a `read` tool that finds the call. `f` may return `Bool`, or a list of what the call made, found again; then the first item becomes the call's answer (the id of the ticket created, say). A `write once` that applies a list of items one by one declares `batch items`; its `verify` returns the items already applied, and the runtime resends only the rest. `requires` sends preconditions to the tool's server, which checks them in the same step as the effect.

Policies. A `policy` for a tool states rules the runtime checks before every call, from a graph or from an agent: `require amount > 0`; `require order from get_ticket.order`, the argument must be a value a tool returned in that field, not one that only appears in a text the model read; `deny in agent if amount > 100`. A refused call is not made; in an agent it becomes an observation for the model. A tool a policy denies to agents outright cannot be given to one (`E0724`).

Other constructs, each with its own compiler rules: `loop` and `rounds` with a mandatory bound; `for each` (fan-out); `race` between strategies, cancelling the losers and undoing what they wrote (a tool may declare `compensate g(...)`, a keyed write that undoes it: a saga, kept in the journal); `agent` (the ReAct loop [24]), which only gets tools without irreversible effects; `entity`, state shared across runs with pure handlers; `receive`, a durable wait for an outside message with a mandatory deadline; and a model router that tries the cheapest model first.

4 What the Compiler Checks

The checks use three things: the type of each value, the effect of each step and the dependency graph. Table 2 lists those about state.

Code	Rejects or warns about
`E0304`	`write once` with no policy for uncertain outcomes
`W0601`	`write` without an idempotency key
`W0602`	Two external writes with no defined order
`W0603`	A `send` to an entity computed from an `ask` to it, when the handler stores a new value (lost update)
`W0604`	A race branch that writes with no `compensate`
`E0695/6`	A compensation that is not a keyed write, or takes other values than the call's
`W0605`	A `write once` in a loop whose arguments do not change between turns
`E0640`	An agent given a `write once` tool
`E0671`	`receive` without a deadline
`E0637`	A `batch` that is not a list, or not verified
`W0701–3`	A declaration the tool's MCP server contradicts (`check --tools`)

Table 2: State checks (partial; the specification lists all).

Several came from the evaluation: `W0605` from the real-bug study (§6.6), `E0637` and the list form of `verify` from LIMBO (§6.7), and `W0701–3` from the observation that a wrong declaration is the weakest point (§6.8).

4.1 What the rules guarantee

We formalize a core of the language: calls to models, `read`, `write key p`, unkeyed `write`, and `write once` π; a journal split into what is on disk and what only the operating system holds; transport faults (lost request, lost response); and process or machine crashes. Resuming is running the program again with the same journal. The rules are:

- **R0.** A call whose journal entry is `done(κ, v)` returns `v` without being made.
- **R1.** A read or model call is made, retrying temporary errors, and recorded without syncing.
- **R2.** A keyed write syncs the journal, then sends with key `k` = ⟦p⟧, retrying with the same key, and records `done` synced.
- **R3.** A `write once` records `begin(κ)` synced, sends, and on an uncertain error applies its policy **D1**: accept, verify (done if found, resend if not), or pause for a person.
- **R4.** On resume, `begin(κ)` without `done` is uncertain and goes to D1.

The theorems hold under five hypotheses: (H1) the journal honours `fsync`; (H2) a service declared with a key honours it; (H3) `verify` is fresh: it sees every committed write, and no request in flight commits after it reads; (H4) the transport delivers each request at most once; (H5) a person resuming a pause decides according to what happened.

T1 (at most once). Under H1–H5, for any sequence of transport faults, process or machine crashes and resumes, each `write once` and each keyed write is applied at most once per journal key.

T2 (exactly once on completion). If the run finishes, each keyed write and each `write once` with `verify` or `pause` was applied exactly once (at most once with `accept_loss`).

T3 (nothing done is redone). A call with `done` on disk is never sent again, and resuming uses its recorded answer. Only H1 is needed.

For one call, the proofs are mechanized in Lean 4 (`formal/Effects.lean`, checked in CI): `write once` and the keyed write are transition systems in which every choice of the world is a step, and an inductive invariant gives T1–T3 for every execution; two counterexamples show the duplicate when the runtime does not sync. For whole programs the argument is a sketch: T1 follows from the `begin` record reaching the disk before any send, so every resend passes through D1, which resends only when nothing was applied (H3, H5); for keyed writes, R2′s sync makes the key a function of durable state, so it is the same in every attempt, and H2 does the rest. We also check the theorems exhaustively on a bounded executable model of the rules (Table 3): a seven-call program with a model, keyed writes (one whose key comes from the model's answer), `write once` with each policy and a read, under every combination of up to two transport faults and two crashes, with every resume.

Case	Runs	Violating
Hypotheses hold, process crashes	53,166	**0**
Hypotheses hold, machine crashes too	217,928	**0**
No sync before a keyed write (bug fixed)	257,488	18,500
`begin` not on disk, machine crashes	268,360	49,346
Service ignores keys (no H2)	53,166	24,524
Stale read-back, late commit (no H3)	101,662	11,248
Transport redelivers (no H4)	95,556	24,856

Table 3: Bounded exhaustive check of T1–T3 (`bench/formal/model.py`).

The last three rows are, in the model, exactly the three causes of duplicates LIMBO measured on Calyx (§6.7). Writing rule R3 with H1 explicit also exposed two holes in the runtime, now fixed: a `write once` went out even if its `begin` record failed to reach the disk, and a keyed write did not sync the journal first, so after a machine crash a model asked again could produce a different key (the third row).

5 The Runtime

The compiler emits an intermediate representation in JSON that a runtime written in C executes, with an I/O layer in Rust (model APIs, MCP clients, sandboxes).

Per-call journal. Every call has a stable key, its place in the realized graph (step, loop turn, `for each` item), and every answer goes to an append-only journal. Before any external write, the journal is synced to disk; a `write once` also records `begin` first, so finding `begin` without an answer means the outcome is uncertain and the tool's policy decides.

The key reaches the server. The idempotency key and the preconditions (`requires`) travel to the tool's server in the MCP call's metadata, so even a repetition caused by a runtime bug does not pay twice. A server in front of another service reports that service's temporary errors with an error text starting `Timeout:` or `Unavailable:`, and a `write once` then applies its policy instead of failing.

Entities. An entity's state lives in a file, changed by one message at a time per key (`flock`), with the id of each applied message stored with it: a message is never applied twice, even if the run dies between the entity applying it and the journal recording it.

Waits. A `receive` records its absolute deadline once; the run exits (state `waiting`) and a scheduler (`calyx tick`, run from cron) resumes it when the message arrives or the deadline passes. The arrival time is recorded, and a late answer does not count even if nobody resumed the run yet (§6.3).

Declarations against servers. MCP servers may annotate tools with hints (`readOnlyHint`, `idempotentHint`) [14]. On each tool's first call, and in `calyx check --tools` without running, Calyx compares them with the declaration and warns on a contradiction (§6.8).

Several machines. With the journal in PostgreSQL, the same lines go to a table, each committed before the next call; a run is executed under a session advisory lock on its id, which PostgreSQL drops with the session of a process that dies, so a worker on another machine takes the run over and finishes it from the journal. An entity is a row: a message locks it, runs the handler and writes the new state with the ids of the applied messages in one transaction, so exactly-once holds across machines; the waits and messages of `receive` are rows too.

Programs as servers. `calyx serve` turns a program into an MCP server: each graph is a tool, annotated from the program (`readOnlyHint` when no step writes, else `idempotencyKeyHint`). The key hint holds by construction: a call with a key is tied to one run, and the same key returns its answer, or resumes it if it died, instead of running it again.

6 Evaluation

We ask: is derived parallelism good (Q1)? Does the compiler catch state bugs Python does not (Q2)? Is recovery correct without hand-written code (Q3)? Is the runtime's cost negligible? The baselines are sequential Python, hand-written asyncio, LangGraph 1.2.12 [11] (default `durability` and `sync`) and Temporal (Python SDK 1.34.0, server 1.32.0) [21]. All run against the same fake world: models with fixed latency and the same MCP tools (a store with payments and e-mail). "Manual care" is the code each system's documentation recommends and a careful programmer writes: idempotency keys, checking before resending, a deadline kept in state. Two experiments were repeated with a real model (Gemini). Code, data and the commands to rerun everything are in the repository (`bench/`).

6.1 Parallelism and cost (W1, E2)

A fan-out of N questions (search + summary each) and a report, at most 8 calls at once. With every model call taking 1 s, Calyx stays within 30–50 ms of the theoretical bound ($\lceil N/8 \rceil + 1$ s), hand-written asyncio within 80–95 ms and LangGraph about 0.8 s behind (mostly start-up). With the real model (Table 4) the picture holds: Calyx ties asyncio. Without latency, Calyx's cost is linear up to 10^5 items, 0.12–0.16 ms per item, with the journal costing up to 14%; asyncio is about 5× cheaper per item (Calyx makes a real MCP call per item) and LangGraph reaches 7 ms per item and grows.

System	N = 5	N = 10
Calyx	**5.1 s**	**6.1 s**
Python asyncio (by hand)	5.2 s	6.4 s
LangGraph	7.3 s	8.1 s
Python, sequential	10.6 s	17.3 s

Table 4: W1 with `gemini-3.5-flash-lite`, median of 3 runs. N is small because of the test key's rate limit.

Measuring cost also found a bug in Calyx: an agent's step was re-evaluated from its first turn on every answer, a cost cubic in the number of turns. Keeping the agent's progress between evaluations fixed it (400 turns: from 70 s to 1.5 s); what remains is quadratic and comes from the chat protocol, which sends the whole conversation every turn.

6.2 Recovery with external effects (W2)

The refund of Listing 1, with the process killed at six points: after each recorded step, and with each effect in flight (the store paid or sent, the answer had not arrived). Correct is one payment, one e-mail and no model call redone.

System	Correct	With care
Calyx	**6/6**	—
Temporal	4/6	6/6
LangGraph	4/6	6/6
`durability="sync"`		
LangGraph default	3/6 (2 model calls redone)	6/6
Python, no checkpoint	2/6	6/6

Table 5: W2, recovery after `kill -9`. Identical, case by case, with the real model.

Between steps, Temporal and LangGraph `sync` are right, like Calyx. With an effect in flight, no per-step record helps, not even Temporal's history: the effect happened and the answer was lost, and the activity is retried after its timeout, as it should be. Only the effect's contract avoids the duplicate (a key the provider honours, or checking before resending), and in the baselines it is optional. With the real model the counts are the same; Calyx resumes in at most 2.7 s, Temporal in 11–16 s with default timeouts.

Against a real service. We repeated the crash matrix against the Stripe API in test mode (`bench/stripe/`): a Stripe refund, whose key Calyx's tool server forwards to Stripe's own `Idempotency-Key`, and a store credit (a customer balance transaction), which Stripe cannot deduplicate and Calyx declares `write once` with `verify`. Killed after each step and with each effect in flight, Calyx made exactly one refund and one credit in all five cases, counted by Stripe itself, in two runs. The same program with the contracts left out got 3/5: one duplicate refund and one duplicate credit, both with the effect in flight.

6.3 Waiting for a person with a deadline (W3)

The refund with human approval and a deadline, the process stopped during the wait, in six scenarios.

Scenario	Calyx	LangGraph	Temporal
Approved	✓	✓	✓
Deadline passes, all stopped	✓	waits forever	✓
Answer sent twice	✓	✓	✓
Late answer, after refusal	✓	pays	✓
Late answer, before resume	✓	pays	pays
On-time answer, resumed later	✓	✓	✓

Table 6: W3. With manual care, LangGraph and Temporal get 6/6.

LangGraph has no deadline for an `interrupt()`. Temporal has one, but with no worker running at the deadline, the timer and the late signal reach the next worker together and the SDK delivers the signal first. **Calyx had the same bug**, found by this measurement and fixed (the delivery time is recorded and checked against the deadline); before the fix it got 5/6.

6.4 Shared memory (W7)

A conversation turn that adds facts to a user's memory, with simultaneous runs and crashes between storing the memory and recording it. Calyx gets 3/3. LangGraph's `Store` loses updates with 20 simultaneous runs (in 3 of 6 rounds, losing 1 to 3 conversations), stores again after a crash and resume, and repeats 30 facts when 10 runs all crash and resume; with manual care (one item per fact, keyed by message id) it gets 3/3.

6.5 Several machines (W8)

With the journal in PostgreSQL, runs move between machines (§5). W8 measures what that costs, against PostgreSQL 16 on the same machine and a few milliseconds away (a proxy that delays every byte; 2.8 and 7.0 ms of round trip), median of 5 (`bench/run_w8.py`).

Journal	Local	2.8 ms	7.0 ms
File	0.12 s	—	—
PostgreSQL, a commit per line (v0.3.5)	0.85 s	6.9 s	15.8 s
PostgreSQL, group commit	**0.15 s**	**0.24 s**	**0.37 s**

Table 7: W8, W1′s fan-out of 500 questions with models that answer at once (1,003 journal lines from 8 threads).

The first version committed every line in its own transaction, under the journal's lock: one round trip per line. A writer thread now commits all the lines appended meanwhile in one transaction, and the runtime waits for it before every external write and at the end, the file journal's rule (`fsync` at most once a second, always before a write). An agent of 100 turns (one thread) goes from 3.5 s to 0.43 s at 7.0 ms; the file takes 0.25 s.

Taking a run over. Machine A dies in the middle of W2′s refund; a `calyx worker` on B finishes it, one payment and one e-mail. When A's process is killed, its OS closes the connection and B has the run 1.3 s later (most of it B's polling). When A's machine goes silent (we drop every packet of its connection, as a power cut would), the server learns it only from TCP keepalive, two hours by default: v0.3.5 did not take the run over in 60 s. Each connection now sets its session's keepalive (5 s, then 3 probes 2 s apart): 11.5 s.

Entities. Eight processes on two machines send 50 messages each to one entity: none is lost (400 of 400). A message is two round trips, the row locked during one: 116 messages/s at 7.0 ms, 620–750 with the database on the machine, 740–880 in files on one machine. Listing 531 runs (`calyx runs`, which the worker does on every pass) takes 0.13 s; the first version read every journal, 0.90 s.

6.6 Bugs before running (Q2)

A corpus of 54 state bugs, each a Calyx program with its expected outcome in its header: the compiler catches 35, 2 cannot be written at all, the runtime catches 11 and 6 escape; 48 of 54 never cause damage. Of the 16 ported to typed Python + LangGraph, pyright and mypy catch 2 of the 14 that Calyx rejects, and LangGraph stops 3 when running, 2 of them after the damage. The corpus and the ports were written by the compiler's author, which is the main threat to this result. A kit for an independent port (E4) is in `bench/e4_porting/`; it has not been run yet.

6.7 Real bugs

To avoid depending on bugs written by the author, we searched the issues of LangGraph, CrewAI and AutoGen [23] with eight searches fixed before reading the results, and classified each report by what Calyx would do with the same workflow, taking the class least favourable to Calyx when in doubt. Of 79 issues, 44 qualified.

Class	Issues
Bug in the framework itself (neutral)	29
Calyx avoids it at runtime	5
Cannot be written in Calyx	2
Calyx lets it through	**8**
Compiler catches it	**0**

Table 8: 44 real bugs from LangGraph, CrewAI and AutoGen issues.

The compiler caught none. Issues report the framework misbehaving, not the programmer's mistake: whoever forgets the idempotency key does not open an issue, they find the double payment in production. Public issues do not measure Q2 well, and they do not confirm it. The study did find a real gap (a `write once` inside a retry loop pays on every turn), which became warning `W0605`; the counts above are for Calyx before the study.

6.8 An external benchmark: LIMBO

LIMBO [12] injects faults into six simulated services and checks, in a ledger, what each one actually did. We wrote its 12 tasks as Calyx programs, declaring the tools only from the documentation an agent sees, and ran them on its E2 grid (205 episodes in which the fault fired) with LIMBO's own fault injector and grader, unchanged. There is no model: each task is a fixed program, so what is compared is recovery. The models' numbers are the ones LIMBO publishes.

	EOS	Dup.	TS
Calyx, native contract	**76%**	**24%**	**100%**
3 frontier models, vanilla	74–79%	20–26%	99.5–100%
Best contract-aware harness	77%	23%	100%
Outcome oracle	88%	12%	100%
Calyx, key on every write	**100%**	**0%**	**100%**

Table 9: LIMBO E2 grid. EOS: exactly-once success; Dup.: episodes with a duplicate; TS: task success.

Under the native contract (only payments and one social platform accept a key), Calyx matches the best models and does not beat them. All its duplicates come from two fault modes no client can fix without a key: transport redelivery (74% for every system, the oracle included) and late commits, where the write is still in flight when the read-back looks (71%; the best harness has 69%). Where a read-back resolves the fault, Calyx never duplicated. When every write accepts a key, duplicates vanish by construction: the key is in the tool's declaration, not in the model's choice at each call. This supports, from the outside, LIMBO's conclusion that the guarantee lives in the tool contract; Calyx's contribution is to make that contract written, checked and the same in every run, at no token cost.

LIMBO found four gaps in Calyx, all fixed before these numbers: `verify` could only say whether a write happened, not return what it made (now it can, which avoided a duplicate in 40 of the 205 episodes); a tool server could not report that the service behind it timed out; a batch write cut in half could not finish (now `batch`); and a person resuming a pause could not give the tool's answer (now `--uncertain done=<answer>`). The batch now verifies instead of pausing, which costs something: under a late commit it duplicates like every other verified write.

6.9 Wrong declarations

The weak point of the approach is the declaration itself: if the programmer declares an effect wrongly, the compiler believes it. We declared each of LIMBO's 22 tools each possible way (`read`, unkeyed `write`, keyed `write`, `write once`) and classified each declaration with LIMBO's internal contracts. A declaration is dangerous if the runtime may repeat a non-idempotent write.

Dangerous declaration	Warned
Non-idempotent write declared `read`	10 of 10
Non-idempotent write as unkeyed `write`	10 of 10
Keyed `write` on a service that ignores keys	**0 of 8**

Table 10: What MCP annotations catch. 3 of 60 safe declarations were also warned (idempotent writes declared `read`).

The annotations catch treating a write as a read and forgetting the key. They miss the mistake that mattered most in LIMBO, trusting a key the service ignores, because MCP annotations have no vocabulary for idempotency keys [14]. We propose one to MCP, `idempotencyKeyHint`, per tool (`docs/mcp/idempotency-key-hint.md`): with our LIMBO adapter declaring it from which services honour a key, Calyx warns (`W0703`) on all 8 such declarations, with no new false positives. The adapter, not LIMBO, declares it: this shows what the check does when servers tell the truth. When they may not, `calyx check --tools --probe` tests the claim in a service's test environment: two calls with the same key, then a count of the effects with a read tool. Against Stripe in test mode it passes our server, and flags (`E0704`, two refunds) a version of it that stops forwarding the key while still claiming `idempotencyKeyHint: true`.

6.10 Prompt injection and policies (E6)

A support agent with six tools (ticket, order, customer, refund, e-mail, address) handles 18 tickets: 6 benign, 12 with an attacker's instruction in the ticket or in the order's notes (refund 900, refund another order, mail the customer's data out, change the address, look up another customer, mail a phishing link). Both programs have the same prompt, with the rules written in it; one adds four policies (Listing 2). The tools server records every call it receives.

```
policy refund:
    require amount > 0
    require order from get_ticket.order
    deny in agent if amount > 100
policy send_email:
    require to from get_order.email
    require not ("://" in body)
```

Listing 2: Two of the four policies of E6.

Model, rules	Attacks	Benign done	Refused
Obedient, in the prompt	12 of 12	6 of 6	0
Obedient, `policy`	**0 of 12**	6 of 6	10
Gemini, in the prompt	0 of 24	12 of 12	0
Gemini, `policy`	0 of 24	12 of 12	0

Table 11: Attacks that happened. Obedient: a fake model that runs every `CALL tool {...}` line it reads, the worst case. Gemini: `gemini-3.5-flash-lite`, 2 repetitions. No legitimate call was refused.

With the obedient model, rules in the prompt stop nothing and policies stop every attack (Table 11); the two address changes are not even attempted, since the agent with policies cannot have that tool. Gemini fell for none of the 24 attempts with the rules only in the prompt, and with policies it never tried a call they refuse: the policies cost nothing (median 5.1 s against 4.6 s per ticket, within the provider's variation) and did nothing. In both versions, the same two attacked tickets (mail the data out; a phishing link, both in the ticket's text) made Gemini drop the legitimate refund too: no harm, but no service, and policies cannot prevent that. So E6 does not show that a current model needs policies against these attacks, which are one sentence each; it shows that what policies declare holds whatever the model does.

6.11 Writing a real workflow

A customer-support example (typed triage, an agent with read-only tools, refunds with approval and deadline, customer memory) ran with Gemini on all three paths and exposed two language problems, both fixed: the wait for approval started before the proposal existed, and there was no way to show the person what they were approving (now `receive ... about`).

It also exposed a limit the language does not address: the model promised actions it did not take.

7 Threats to Validity

Same author on both sides. The baselines, the bug corpus and the LIMBO programs were written by Calyx's author. LIMBO's tasks, faults, grader and published baselines are by others, and the LIMBO adapter only uses LIMBO's public tools; the independent port (E4) is pending. **Proofs.** T1–T3 are mechanized in Lean only for one call; for whole programs they have proof sketches and a bounded exhaustive check. Both models are written by hand, not extracted from the code. **Bugs found in Calyx.** Several Calyx bugs were found by measuring and fixed before the numbers reported; we say where. **Wrong declarations.** The check against MCP annotations only works for servers that send them, and misses ignored keys. **Models.** Most experiments use fake models; W1 and W2 were repeated with Gemini at small N, and in W2 the model always proposed the same amount, so the risk of a different answer after a crash was not exercised. The E6 attacks are fixed sentences, not adapted to the model. **Setting.** One machine; W8 (several machines) adds latency with a proxy rather than running against a database across a real network; only the Stripe runs use a real service over the network, the rest use fake services; versions are pinned. **Real-bug study.** Issues read from their summaries, one classifier, a sample limited by GitHub search.

8 Related Work

Duplicated effects in agents. LIMBO [12] measures where the exactly-once guarantee should live and finds it depends on the tool contract. Other studies count idempotency violations under retries [8] and evaluate recovery from ambiguous tool outcomes [19]; a tool wrapper with post-condition verification, idempotency keys and verify-before-retry reduces duplicates [13], the same mechanism as Calyx's `verify` and keys, offered as an optional library. Calyx starts from the same conclusion and makes the contract mandatory and compiler-checked.

Transactions and runtime enforcement. GoEX [16] argues for undo and damage confinement; SagaLLM [4], Atomix [15], Cordon [5] and Agentic Transaction [20] provide transactional or compensating execution; AgentRewind [25] checkpoints and rewinds agents; AgentSpec [22] enforces user rules at runtime. These are runtime mechanisms; Calyx asks the program to declare what they need and checks it before running. Its compensations (`compensate`) are such a saga, limited to race losers and required to be keyed writes. Its policies are AgentSpec-like rules, checked by the compiler for type, purity and provenance and, when the arguments are written in the program, already at compile time (`E0723`).

Static analysis and calculi for agents. Recent work analyses agent programs written in existing frameworks for structural properties [1] and non-termination [2], gives a typed calculus for agent composition [3], or tracks information flow in LLM programs [7]. CaMeL [6] defends against prompt injection by separating control from data flow and tagging values with capabilities; Calyx's `from` is a narrower, declared case of that idea, checked per argument. Calyx checks a different property, the contract of external effects, which existing frameworks do not write down anywhere; its formal core could extend such a calculus with journals and crashes.

Frameworks and durable execution. ReAct [24] interleaves reasoning and acting in a model-driven loop, which Calyx offers as a bounded construct; AutoGen [23] organizes agents as conversations; DSPy [10] optimizes prompts of typed pipelines; LangGraph [11] makes the graph explicit and checkpoints each step. Temporal [21] and Restate [17] journal each step and resume without redoing it; in both, the idempotency of an external effect is the programmer's responsibility. Calyx ties Temporal on recovery when the care is written, and differs by requiring it.

9 Conclusion

Measured against LangGraph and Temporal on crashes, waits and concurrency, the result is consistent: the baselines are right when the programmer writes the care, and Calyx requires it to be written. On an external benchmark, Calyx ties the best models when tools do not accept keys and reaches zero duplicates when they do, which supports the claim that the guarantee lives in the contract and that requiring it is worthwhile. The runtime's cost is small and its parallelism matches hand-written asyncio. The weakest part of the evidence is the one that matters most for the thesis: whether the compiler catches, before running, the bugs real programmers make. The author's corpus says yes; public issues say neither yes nor no; an independent port is the next step.

Availability. Calyx, the experiments and their data are in the repository https://github.com/daltonfontes/calyx, whose `bench/` directory has the scripts that produce every number in this paper.

References

[1] 2026. Agentproof. Retrieved from https://arxiv.org/abs/2603.20356

[2] 2026. When Agents Do Not Stop. Retrieved from https://arxiv.org/abs/2607.01641

[3] 2026. λ_A, a typed calculus for agent composition. Retrieved from https://arxiv.org/abs/2604.11767

[4] Edward Y. Chang and Longling Geng. 2025. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. Retrieved from https://arxiv.org/abs/2503.11951

[5] Zheng Chen, Hanqing Liu, Duling Xu, Dong Dong, Jialin Li, Bangzheng Pu, and Jidong Zhai. 2026. Cordon: Semantic Transactions for Tool-Using LLM Agents. Retrieved from https://arxiv.org/abs/2606.17573

[6] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. Retrieved from https://arxiv.org/abs/2503.18813

[7] Zac Garby, Andrew D. Gordon, and David Sands. 2026. The LLMbda Calculus. Retrieved from https://arxiv.org/abs/2602.20064

[8] Sanjana Gopnal Swamy. 2026. Do LLM Agents Act Exactly Once? Measuring Idempotency Violations Under Retries. Retrieved from https://github.com/gssanjana4/idempotencybench

[9] Pat Helland. 2012. Idempotence Is Not a Medical Condition. *ACM Queue* 10, 4 (2012), 30–46. https://doi.org/10.1145/2181796.2187821

[10] Omar Khattab and others. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations (ICLR)*, 2024.

[11] LangChain. 2026. LangGraph. Retrieved from https://github.com/langchain-ai/langgraph

[12] Jiapeng Li. 2026. LIMBO: Benchmark code, harness adapters and episode traces. Retrieved from https://github.com/jaxblack/limbo-bench

[13] Isham Kalappurackal Mansoor, Abhishek Phadke, and Pratip Rana. 2026. Verified Tool Calls Improve LLM Agent Reliability Under Non-Atomic Failures. Retrieved from https://arxiv.org/abs/2608.02645

[14] Model Context Protocol. 2025. Model Context Protocol Specification, Version 2025-06-18: Schema Reference (ToolAnnotations). Retrieved from https://modelcontextprotocol.io/specification/2025-06-18/schema

[15] Bardia Mohammadi, Nearchos Potamitis, Lars Klein, Akhil Arora, and Laurent Bindschaedler. 2026. Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows. Retrieved from https://arxiv.org/abs/2602.14849

[16] Shishir G. Patil, Tianjun Zhang, Vivian Fang, Noppapon C., Roy Huang, Aaron Hao, Martin Casado, Joseph E. Gonzalez, Raluca Ada Popa, and Ion Stoica. 2024. GoEX: Perspectives and Designs Towards a Runtime for Autonomous LLM Applications. Retrieved from https://arxiv.org/abs/2404.06921

[17] Restate. 2026. Restate. Retrieved from https://restate.dev/

[18] Stripe. 2026. Idempotent requests. Retrieved from https://docs.stripe.com/api/idempotent_requests

[19] Shengyao Sun. 2026. Did It Happen? Counterfactual Evaluation of LLM Agent Recovery from Ambiguous Tool Outcomes. https://doi.org/10.21203/rs.3.rs-10730245/v1

[20] Zhaoyan Sun, Xiaoxiao Wang, and Guoliang Li. 2026. Agentic Transaction: Towards ACID-Compliant Agent Systems. Retrieved from https://arxiv.org/abs/2608.13900

[21] Temporal Technologies. 2026. Temporal. Retrieved from https://temporal.io/

[22] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. In *Proc. 48th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. 2938–2950.

[23] Qingyun Wu and others. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Conference on Language Modeling (COLM)*, 2024.

[24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.

[25] Yu Zhuang, Kefei Chen, Yitong Duan, Shuxin Zheng, Jian Li, and Xu-Yao Zhang. 2026. AgentRewind: Recoverable Execution for Long-Horizon LLM Agents. Retrieved from https://arxiv.org/abs/2608.14380